Pontificia Universidad JAVERIANA Colombia

Tools y primer agente

Semana 3 · Function calling con la cartera de ACME
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3

RAG y evaluación
Semanas 4–6

Model Context Protocol
Semana 7

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 3 · Tools y primer agente

- Lo que el modelo no sabe: datos de otro sistema
- De función a herramienta con `@tool`
- El ciclo modelo → herramienta → modelo
- `create_agent`: el ciclo repetido por nosotros
- Errores como datos y respuestas comprobables
- Una API externa: la TRM del día

Las ideas que traemos de la semana 2

Repaso

Antes de avanzar

1. Una plantilla Jinja adapta el prompt a los datos.
2. `StrictUndefined` detecta un dato olvidado antes de llamar.
3. Pydantic valida la forma; la fidelidad requiere otra comprobación.
4. Las trazas muestran qué recibió el modelo y qué devolvió.

Lo que el modelo no sabe

El problema de la semana

Un dato de otro sistema

«Soy el cliente C-1042. Si pago **500.000 pesos** hoy, ¿quedo al día en mi Crédito Libre Inversión?»

Claude puede leer el mensaje, pero **no sabe cuánto debe el cliente**. Ese dato vive en la cartera de créditos.

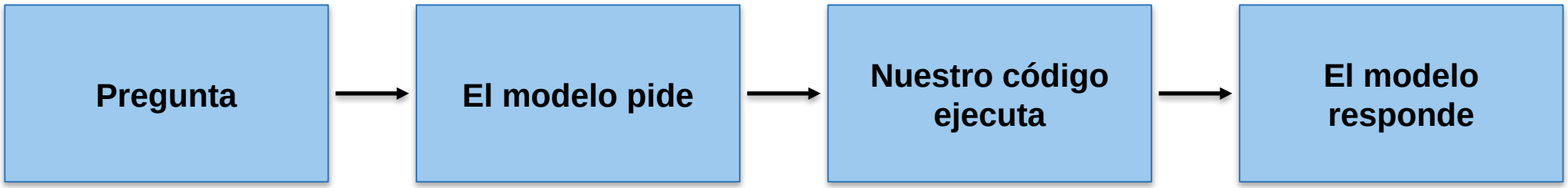
Pontificia Universidad JAVERIANA Colombia

Tool calling

El modelo propone; nuestro código ejecuta

El ciclo de una herramienta

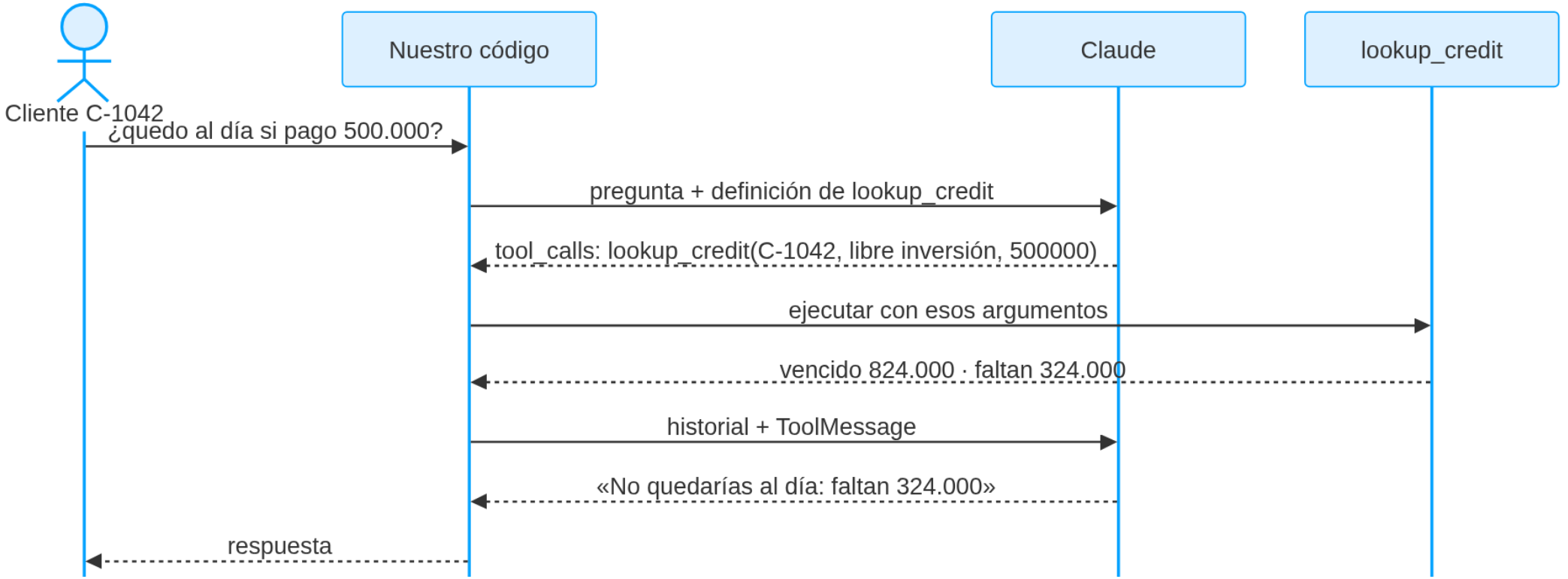
Function calling, tool calling o tool use



El modelo nunca ejecuta la función: solo propone cuál llamar y con qué argumentos.

El ciclo, mensaje por mensaje

L04 · diagrama de secuencia



Dos llamadas al modelo. El modelo propone la llamada; nuestro código la ejecuta y devuelve un `ToolMessage`; con eso el modelo responde.

De función a herramienta

L04 · secciones 1 y 2

El contrato

Nombre, docstring y tipos son lo que el modelo lee.

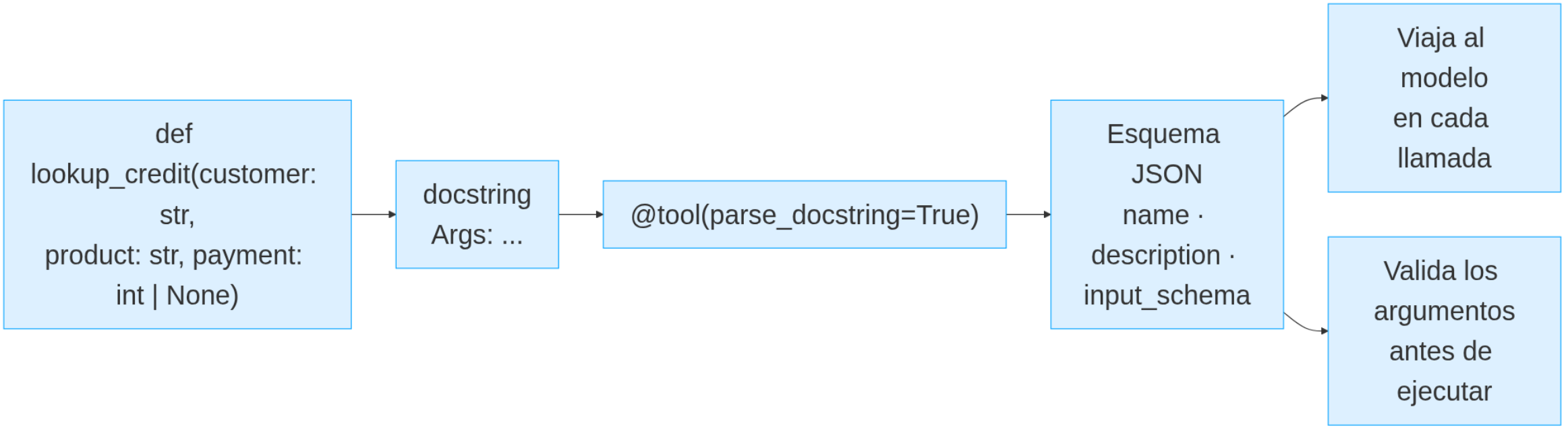
```
def lookup_credit(customer: str, product: str,
                  payment: int | None = None) -> dict:
    """Consulta el estado de un crédito de un cliente de ACME.

    Args:
        customer: código del cliente, por ejemplo C-1042.
        product: código (CLI, CEX) o palabras del producto.
        payment: pago en pesos que el cliente quiere hacer.
    """
    ...

credit_tool = tool(lookup_credit, parse_docstring=True)
```

De la función al esquema

Lo que el modelo ve de una herramienta



El contrato. Nombre, docstring y tipos se convierten en un esquema JSON: guía al modelo y valida los argumentos antes de ejecutar.

El modelo pide la herramienta

L04 · secciones 3 y 4

Cerrar el ciclo a mano

Dos llamadas al modelo y una a nuestra función.

```
model_with_tools = model.bind_tools([credit_tool])
ai_message = model_with_tools.invoke(question)
ai_message.tool_calls
# [{'name': 'lookup_credit',
#   'args': {'customer': 'C-1042',
#            'product': 'libre inversión',
#            'payment': 500000}, ...}]

tool_message = credit_tool.invoke(ai_message.tool_calls[0])
messages = [HumanMessage(question), ai_message, tool_message]
model_with_tools.invoke(messages)
# «No quedarías al día: el vencido es 824.000 COP...»
```

Lo que devuelve la cartera

La herramienta, sin modelo

Campo	C-1042 · CLI · pago 500.000
`balance_cop`	8.450.000
`overdue_cop`	824.000 (2 cuotas de 412.000)
`days_past_due`	45
`covers_overdue`	false
`missing_cop`	324.000

Pontificia Universidad JAVERIANA Colombia

El primer agente

Un ciclo que se repite hasta tener la respuesta

create_agent

L04 · sección 5

El ciclo, automatizado

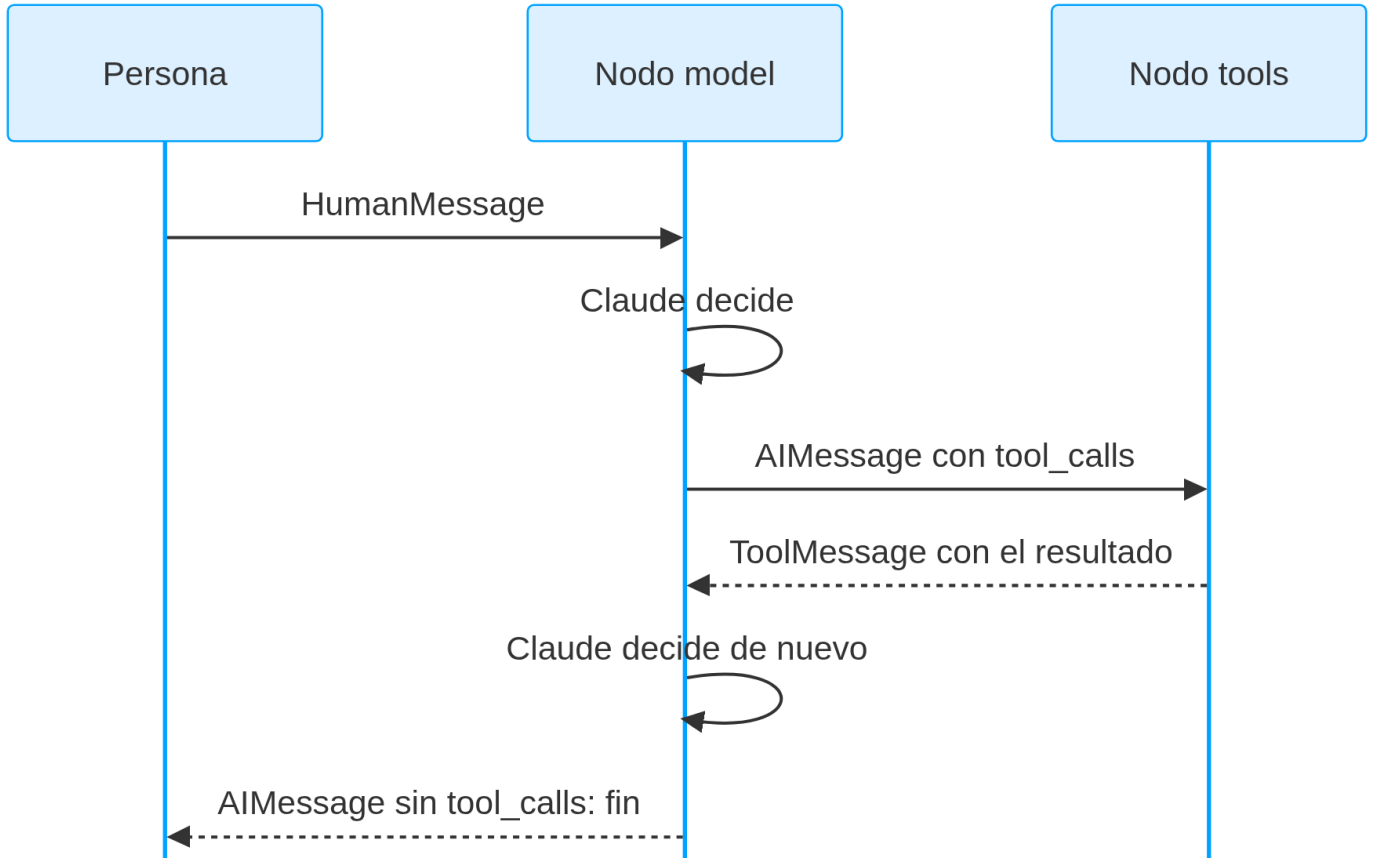
Un agente es
un modelo que usa
herramientas en un ciclo.

```
from langchain.agents import create_agent

agent = create_agent(
    model,
    tools=[credit_tool],
    system_prompt="Eres el asistente de soporte de ACME. "
                  "Usa solo las cifras de las herramientas.",
)
result = agent.invoke(
    {"messages": [HumanMessage(question)]})
result["messages"][-1].text
```

El bucle de create_agent

Diagrama de secuencia



model ⇄ tools. El agente repite modelo → herramienta → modelo mientras la respuesta traiga `tool_calls`; sin ellas, termina.

Cuando la herramienta no encuentra el dato

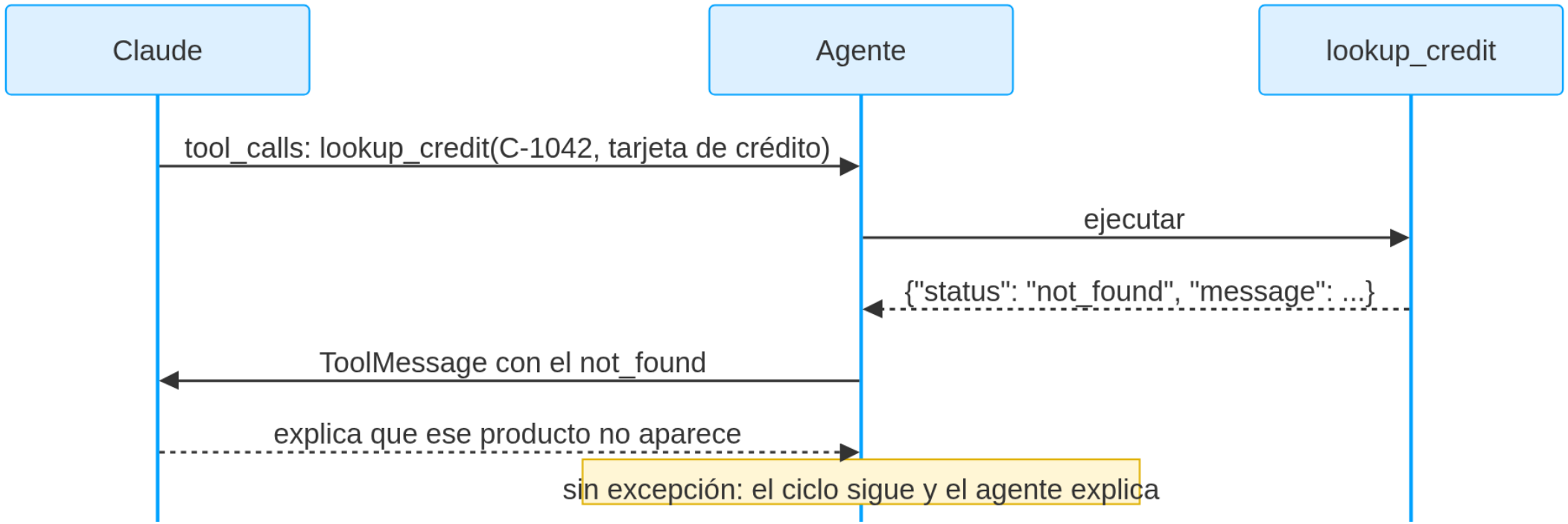
L04 · secciones 6 a 8

Errores como datos

1. La herramienta devuelve `{'status': 'not_found', 'message': ...}` en lugar de lanzar un error.
2. El agente puede explicarle al cliente qué no encontró.
3. Con varias herramientas, el agente elige cuál pedir según su descripción.
4. `tool_choice` puede forzar una herramienta, aunque el modelo no siempre lo respeta.

Un error como dato

L04 · sección 6



not_found. Una excepción cortaría el agente sin respuesta; un `not_found` deja que el modelo le explique al cliente qué falta.

Comprobar la respuesta

L04 · sección 9

Confiar con evidencia

1. Leer los `ToolMessage` del historial: qué cifras devolvió la herramienta.
2. Comprobar que la respuesta usa esas cifras y no otras.
3. Una respuesta convincente sin llamada a la herramienta es una alerta.

Cómo diseñar una herramienta

Principios que usamos en todo el curso

Principio	Por qué
Una tarea por herramienta, con un nombre claro	El modelo elige por el nombre y la descripción
La docstring lista los valores aceptados	Es todo lo que el modelo sabe de los argumentos
Tipos estrictos en los argumentos	Un valor inválido falla antes de ejecutar
Solo lectura, salvo aprobación humana	El modelo puede pedir cualquier cosa
Errores como datos: `status` + `message`	El ciclo sigue y el agente explica
Devolver solo lo necesario, con su fuente	Menos tokens y respuestas comprobables

Pontificia Universidad JAVERIANA Colombia

Una API externa

La TRM del día en datos.gov.co

La API como herramienta

L05 · secciones 1 y 2

Red, tiempos y errores

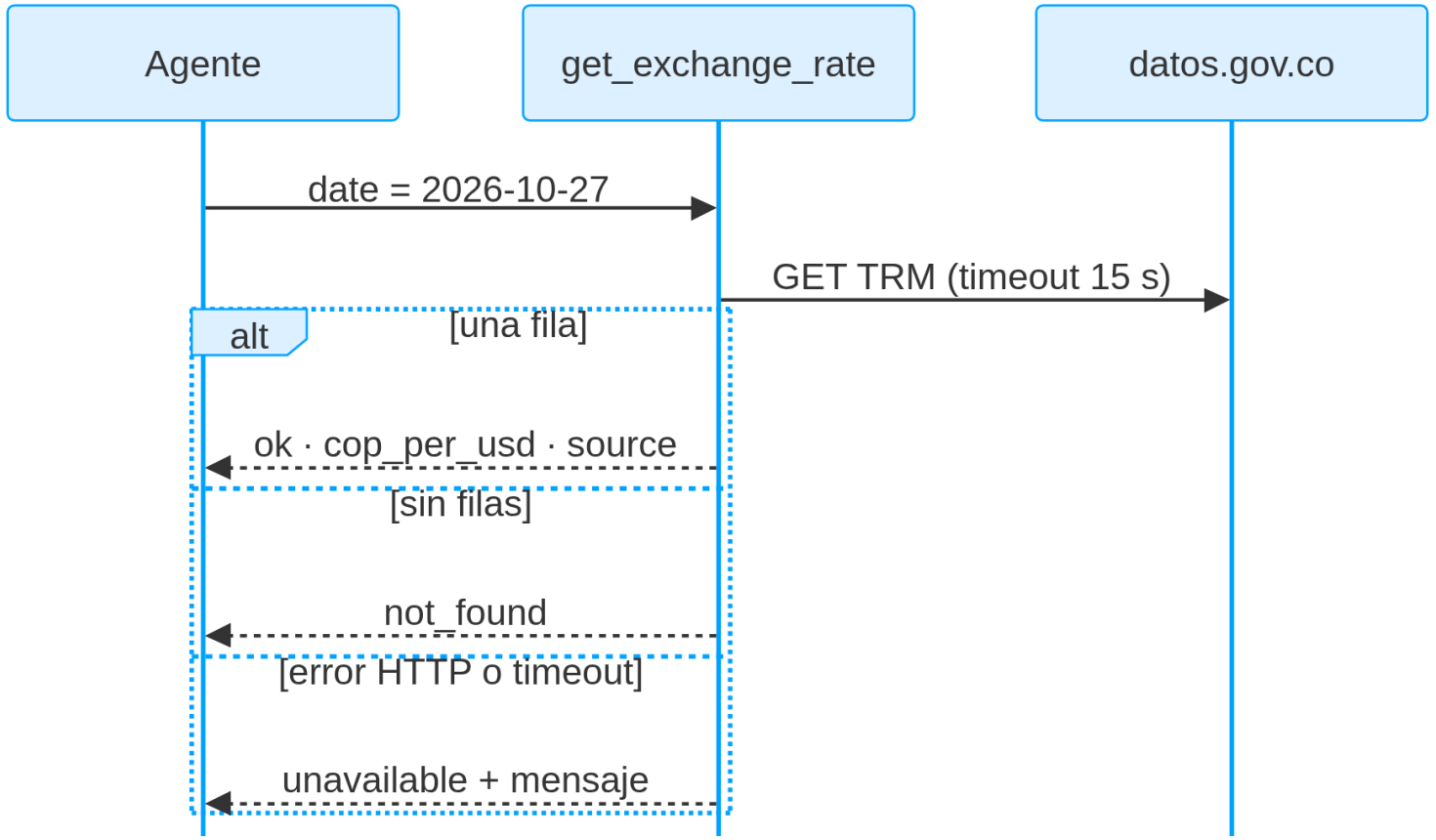
Para el modelo, una API
es igual que
un diccionario.

```
@tool(parse_docstring=True)
def get_exchange_rate(date: str) -> dict:
    """Consulta la TRM que rige en una fecha.

    Args:
        date: fecha en formato AAAA-MM-DD.
    """
    try:
        response = httpx.get(TRM_URL, params=..., timeout=15)
    except httpx.HTTPError as error:
        return {"status": "unavailable", "message": str(error)}
    ...
    return {"status": "ok", "cop_per_usd": 3157.94, ...}
```

Una API con todos sus casos

L05 · diagrama de secuencia



Red, tiempos y errores. Cada desenlace de la red llega al agente como un dato con `status`: ok, not_found o unavailable.

Datos que cambian cada día

L05 · secciones 3 a 5

Comprobar sin cifra fija

1. La TRM cambia cada día hábil: no hay una respuesta fija que esperar.
2. Comparamos la respuesta con la cifra que devolvió la herramienta en esa ejecución.
3. 300 USD × TRM: la cuenta la hacemos también en código.
4. Una condición Jinja del prompt cambia la conducta: usar la TRM de hoy o preguntar la fecha.

Diccionario local frente a API pública

Lo que cambia dentro de la herramienta

Aspecto	Datos en el notebook	API de la TRM
Latencia	Inmediata	Depende de la red: `timeout`
Fallos	Solo datos ausentes	Errores HTTP, caídas, consultas mal formadas
Frescura	Fija	Cambia cada día hábil
Comprobación	Cifra esperada fija	Comparar con lo que devolvió la herramienta
Credenciales	Ninguna	Ninguna aquí; si las hubiera, fuera del prompt

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L04-Tools · L05-ToolsAPI

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. L04: convertir la consulta de la cartera en herramienta y ver `tool_calls` (9 llamadas).
2. L04: cerrar el ciclo a mano y luego con `create_agent`.
3. L04: varias herramientas, errores y comprobación de cifras.
4. L05: llamar la API de la TRM sin modelo y después como herramienta (5 llamadas).

¿Cómo sabremos que funcionó?

Criterios de logro

La prueba del ejercicio

1. Podemos señalar en el historial cada llamada y su resultado.
2. Las cifras de la respuesta coinciden con las de la herramienta.
3. Un cliente o producto inexistente produce una respuesta honesta.
4. Un fallo de la API llega al modelo como dato, no como excepción.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. El modelo propone; nuestro código ejecuta y controla.
2. La descripción de una herramienta es parte del prompt.
3. Las cuentas se hacen en código; el modelo las explica.
4. Los errores devueltos como datos mantienen vivo el ciclo.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Learning LangChain, cap. 6	Agentes y herramientas
AI Engineering, cap. 6	RAG y agentes: herramientas y planificación
Anthropic, Tool use	Cómo el modelo elige y llama herramientas
Documentación de LangChain	`create_agent` y `@tool`

Pontificia Universidad JAVERIANA Colombia

Próxima semana

Embeddings y **vector stores**: buscar en las políticas de ACME.

Notebooks:

`L06-Embeddings.ipynb`

`L07-VectorStores.ipynb`

`L08-RAG.ipynb`

Sergio Alberto Mora Pardo - H1 2027

GitHub · LinkedIn · Repositorio del curso